

Практика — Урок 2

Робимо окремий репозиторій-піщаницю, щоб нічого не зламати в навчальному проєкті. Усі команди виконуємо в **Git Bash** (або в терміналі PyCharm).

Крок 0. Перевірка налаштувань

```
git --version
git config user.name
git config user.email
```

Якщо ім'я або пошта порожні — задай їх (див. `02_nalashtuvannya.md`).

Крок 1. Створюємо репозиторій

```
cd ~/Desktop
mkdir git_sandbox
cd git_sandbox
git init
git status
```

Питання: що написано в першому рядку `git status` і чому «No commits yet»?

Крок 2. Перший коміт

```
echo "print('Hello Git!')" > main.py
git status                # main.py – untracked (червоний)
git add main.py
git status                # main.py – staged (зелений)
git commit -m "add main.py"
git log --oneline
```

Крок 3. Дивимось на три зони в дії

```
echo "print('другий рядок')" >> main.py
git status                        # modified
git diff                         # видно + рядок, який ще не в stage
git add main.py
git diff                         # порожньо!
git diff --staged                # ось де тепер видно зміну
git commit -m "add second line"
```

Зрозумій різницю між `git diff` і `git diff --staged` — це та сама межа між working directory та staging area.

Крок 4. Скасування змін

```
echo "print('випадкова дурня')" >> main.py
cat main.py                      # рядок є
git restore main.py
cat main.py                      # рядка немає — правку відкотили
```

Крок 5. `.gitignore`

```
mkdir __pycache__ && touch __pycache__/main.cpython-312.pyc
touch secret.env
git status                        # Git пропонує додати сміття

printf '__pycache__/\n*.pyc\n*.env\n' > .gitignore
git status                        # тепер видно лише .gitignore

git add .gitignore
git commit -m "add gitignore"
```

Крок 6. Гілка

```
git checkout -b feature-greeting
git branch                # * біля feature-greeting

echo "print('Привіт з гілки!')" >> main.py
git add .
git commit -m "add greeting from branch"
git log --oneline

git checkout main
cat main.py               # рядка з гілки тут НЕМА – так і має бути
```

Крок 7. Злиття

```
git merge feature-greeting
cat main.py               # тепер рядок є
git log --oneline --graph --all
git branch -d feature-greeting
```

Крок 8. Робимо конфлікт спеціально

```
git checkout -b conflict-demo
echo "print('версія з гілки')" > main.py    # один > – файл перезаписується
git commit -am "branch version"

git checkout main
echo "print('версія з main')" > main.py
git commit -am "main version"

git merge conflict-demo      # CONFLICT!
cat main.py                 # побачиш <<<<<< ===== >>>>>>
```

Вирішуємо:

```
echo "print('фінальна версія')" > main.py   # прибрали маркери, лишили одне
git add main.py
git commit -m "resolve conflict"
git log --oneline --graph --all
```

Крок 9. `git stash`

```
echo "print('недороблене')" >> main.py
git stash                # відклали
git status              # чисто
git stash list          # stash@{0}
git stash pop           # повернули
git restore main.py     # приберемо, щоб не тягнути далі
```

Крок 10. Подорож у минуле

```
git log --oneline        # скопіюй хеш першого коміту
git checkout <хеш>       # подивись, яким був main.py тоді
cat main.py
git checkout main        # повернулись
git revert HEAD          # скасували останній коміт новим комітом
git log --oneline
```

Крок 11. Віддалений репозиторій (потрібен акаунт і SSH-ключ)

```
ssh -T git@github.com   # перевірка ключа
git remote add origin git@github.com:USERNAME/git_sandbox.git
git push -u origin main
```

Далі — на GitHub: створи гілку, зроби PR і злий його. Потім локально:

```
git pull origin main
git log --oneline --graph --all
```

Контрольні питання

1. Чим `git diff` відрізняється від `git diff --staged` ?
2. Куди зникає файл після `git add` , і чи є він уже в історії?
3. Що зробить `git restore` з незакоміченою правкою?
4. Чому `.gitignore` треба комітити, а `.env` — ні?
5. Чим `git revert` кращий за `git reset --hard` у командній роботі?